

7. Bölüm

Diziler

7.1 Dizi Nedir

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Doğal olarak eleman sayısı tam sayıdır (**integer**). Elemanların sırasını tam sayı olan **indis** (**index**) belirtir. Kümelerin (**set**) aksine, aynı elemanlar dizide farklı konumlarda bulunabilir. Elemanlara terim adı da verilir. Örnekler;

- ◇ Örnek olarak elemanları 1'den 10'a kadar tam sayıların karesinden oluşan bir dizi, {1,4,9,...,100} olarak gösterilir.
- ◇ Bir başka örnek olarak {K,İ,T,A,P} dizisi verilebilir. Bu dizi ile {P,A,T,İ,K} dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları yani indisleri farklıdır.
- ◇ {1,3,5,1,2,1,7} dizisinde birden farklı sırada yani indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler kullanılır. Dizinin eleman sayısı, dizinin **uzunluğudur** (**size**) ve dizinin uzunluğu dizi tanımlanırken belirlenir!

7.2 Dizi Tanımlama

C# dilinde **tek boyutlu** (**one-dimensional**), **çok boyutlu** (**multidimensional**) ve **pürüzlü dizi** (**jagged array**) olmak üzere üç farklı dizi tanımlanabilir;

```
veri-tipi[] dizi-kimliği; // Tek boyutlu dizi
veri-tipi[,] dizi-kimliği; // Çok boyutlu dizi, Her bir boyut virgül ile ayrılır
veri-tipi[][] dizi-kimliği; // Pürüzlü dizi
```

Dizi Tanımı

C# dilinde bir dizi tanımlanırken, dizinin tutacağı veri tipi ve eleman sayısı mutlaka belirtilmelidir. Diziler **referans tipli** (**reference type**) nesnelerdir. Tüm referans tiplerde olduğu gibi diziler de ve **new** anahtar kelimesiyle, eleman sayısı belirtilerek, öbek (**heap**) bellekte oluşturulurlar. **Tahsis edilen bellek bölgesinin adresini referans tipli dizi değişkeni tutar.**

Dizilerin;

- ◇ İçinde, **aynı veri tipindeki öğeleri barındırır** ve bu öğeler **bitişik bellek bölgesini** paylaşırlar.
- ◇ Elemanları, temel veri tipleri (**byte**, **short**, **int**, **long**, **float**, **double**, ...) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan yapı **struct** veya gösterici (**pointer**) olabilir.
- ◇ İçinde aynı değer birkaç farklı yerde bulunabilir.
- ◇ Veri tipi, **dizideki elemanlarının veri tipidir**. Bir başka deyişle **elemanların veri tipi dizinin veri tipiyle aynı olmalıdır**.
- ◇ Elemanlarının indisi (**index**) sıra belirttiğinden **her zaman tam sayıdır**.
- ◇ İlk elemanın indisi **0**, son elemanın indisi dizi uzunluğundan (**size**) bir küçük olan tam sayıdır. Bunların dışında bir indis girildiğinde hata alınır.

Aşağıda değişik dizi tanımlama örnekleri verilmiştir;

```
//1. Elşeman Sayısı Bilinen TEK Boyutlu Dizi;
int[] ilkdeğeriolmayandizi = new int[5]; // 5 Elemanlı Tek Boyutlu Dizi
ilkdeğeriolmayandizi = null; // üst satırda ayrılan bellek serbest bırakılır.
```

```
//2. Eleman sayısı ve Eleman Değerleri Bilinen Dizi;
int[] dizi1 = new int[6] { 1, 2, 3, 4, 5, 6 }; // 6 Elemanlı Dizi
int[] dizi2 = { 1, 2, 3, 4, 5, 6 }; // Elemanları Listesi Verilertek Dizi Başlatma
```

```
//3. Biz Dizi Referansı Bildirme:
int[] dizi3; //dizi tanımlandıktan sonra elemanlara ilk değer verilemez!
// dizi3 = { 24, 2, 13, 47, 45 }; //derleme hatası olur!

//4. Üstü Kapalı Olarak Dizi Tanımlama;
var arr1 = new [] { 1, 2, 3 }; // new int[] { 1, 2, 3 }; ile aynıdır
var arr2 = new [] { "one", "two", "three" }; // string[] ile aynıdır
var arr3 = new [] { 1.0, 2.0, 3.0 }; // double[] ile aynıdır
```

7.3 Dizi Elemanlarını Değiştirme

Bir dizi tanımlandıktan sonra tam sayı indislerle elemanlara erişilebilir ve erişilen eleman değeri değiştirilebilir;

```
int[] dizi = new int[] { 0, 10, 20, 30 };
Console.WriteLine(arr[2]); // dizinin 3. Elemanı yazdırılıyor: 20
arr[2] = 100; //dizinin 3.elemanına 100 değeri atandı.
Console.WriteLine(arr[2]); // güncellenmiş elemana yazdırılıyor: 100
```

7.4 Dizi Üzerinde Yinelemeler

Dizi üzerindeki elemanlarının tümüne yineleme (**iteration**) ile erişilebilir. Yineleme indis değişkeni içeren **for** döngüleriyle yapılır.

Tek boyutlu dizilere örnek olarak klavyeden girilen 5 adet tam sayıyı, giriş sırasının tersinden ekrana yazan programını verebiliriz;

```
int[] dizi = new int[5]; // 5 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++) {
    Console.Write($"{indis}.Sayıyı Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
for (int indis = dizi.Length-1; indis >= 0; indis--)
    Console.WriteLine($"{indis}.Sayı:{dizi[indis]}");
/*
0.Sayıyı Girin:10
1.Sayıyı Girin:11
2.Sayıyı Girin:13
3.Sayıyı Girin:45
4.Sayıyı Girin:55
4.Sayı:55
3.Sayı:45
2.Sayı:13
1.Sayı:11
0.Sayı:10
*/
```

Bir başka örnek olarak konsoldan girilen 10 adet sınav notundan, ortalamanın üstünde olanları ekrana yazan programı;

```
int[] dizi = new int[10]; // 10 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++) {
    Console.Write($"{indis}.Notu Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("{0} üzerindeki notlar:", dizi.Average());
for (int indis = 0; indis < dizi.Length; indis++)
    if ((double)dizi[indis] > dizi.Average())
        Console.WriteLine($"{indis}.Not:{dizi[indis]}");
/*Program Çıktısı:
0.Notu Girin:10
1.Notu Girin:15
2.Notu Girin:25
3.Notu Girin:35
4.Notu Girin:40
5.Notu Girin:60
```

```

6.Notu Girin:15
7.Notu Girin:20
8.Notu Girin:12
9.Notu Girin:16
24,8 üzerindeki notlar:
2.Not:25
3.Not:35
4.Not:40
5.Not:60
*/

```

Bir başka örnek olarak değeri belli 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani **sapma** (**deviation**) ve karesi alınmış sapmalar yani **varyans** (**variance**) ile standart sapmayı yazdıran program verilebilir.

```

// 10 elemanlı tek boyutlu bir tamsayı dizisi:
decimal[] satislar = { 100.0M, 850.0M, 500.0M, 600.0M, 750.0M, 650.0M, 450.0M, 800.0M, 900.0M,
    ↪ 110.0M };
decimal ortalama = satislar.Average();
Console.WriteLine($"Ortalama:{ortalama}");
double varyansToplam = 0;
for (int indis = 0; indis < satislar.Length; indis++) {
    decimal sapma = satislar[indis] - ortalama;
    decimal varyans = sapma * sapma;
    Console.WriteLine($"Sapma:{sapma}, varyans:{varyans}");
    varyansToplam += (double) varyans;
}
double standartSapma = Math.Sqrt(varyansToplam / satislar.Length);
Console.WriteLine($"Standart Sapma:{standartSapma}");
/*
Ortalama:571,0
Sapma:-471,0, varyans:221841,00
Sapma:279,0, varyans:77841,00
Sapma:-71,0, varyans:5041,00
Sapma:29,0, varyans:841,00
Sapma:179,0, varyans:32041,00
Sapma:79,0, varyans:6241,00
Sapma:-121,0, varyans:14641,00
Sapma:229,0, varyans:52441,00
Sapma:329,0, varyans:108241,00
Sapma:-461,0, varyans:212521,00
Standart Sapma:270,49768945408755
*/

```

Ortalamaya olan uzaklıklar yani sapma verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha çok yakın gruplaştıklarını gösterir.

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan **tekil** (**unique**) olanlarını bulan program verilebilir.

```

int[] dizi = new int[5]; // 5 elemanlı tek boyutlu bir tamsayı dizisi
for (int indis = 0; indis < dizi.Length; indis++) {
    Console.Write($"{indis}.Sayıyı Girin:");
    dizi[indis] = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("Dizi İçindeki Tekil Elemanlar:");
for (int indis = 0; indis < dizi.Length; indis++) {
    int indis2, sayac = 0; // Her elemenda sayacı sıfırla
    for (indis2 = 0; indis2 < dizi.Length; indis2++)
        if (indis != indis2) //elemenin kendisini kontrol etmiyoruz
            if (dizi[indis] == dizi[indis2])
                sayac++;
    if (sayac == 0)

```

```

        Console.WriteLine($"dizi[{indis}]:{dizi[indis]} tekil olarak dizide bulundu.");
    }
    /*
0.Sayıyı Girin:10
1.Sayıyı Girin:15
2.Sayıyı Girin:12
3.Sayıyı Girin:10
4.Sayıyı Girin:12
Dizi İçindeki Tekil Elemanlar:
dizi[1]:15 tekil olarak dizide bulundu.
*/

```

Bir başka örnek olarak 5 elemanlı belli olan dizide en büyük elemana en küçük elemanını ekleyen program verilebilir;

```

int[] dizi = { 10, 12, 3, 4, 6, 12, 4, 7, 8, 9 };
Console.WriteLine("Dizinin İLK Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
Console.WriteLine();
int enKucuk = dizi[0], enBuyuk = dizi[0];
/* İlk elemanlar hem en küçük hem de en büyük olsun. */
int enKucukIndex = 0, enBuyukIndex = 0;
/* En küçük ve en büyük elemanların indisi 0 olsun. */
for (int indis = 0; indis < dizi.Length; indis++) {
    /* En küçük ve en büyük eleman ile indislerini bulan kısım. */
    if (dizi[indis] < enKucuk) {
        enKucuk = dizi[indis];
        enKucukIndex = indis;
    }
    if (dizi[indis] > enBuyuk) {
        enBuyuk = dizi[indis];
        enBuyukIndex = indis;
    }
}
Console.WriteLine($"En Büyük Eleman İndisi: {enBuyukIndex} ve Eleman {enBuyuk}");
Console.WriteLine($"En Küçük Eleman İndisi: {enKucukIndex} ve Eleman {enKucuk}");
//En büyük elemana en küçüğünü ekleme
dizi[enBuyukIndex] += dizi[enKucukIndex];
Console.WriteLine("Dizinin SON Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
/*
Dizinin İLK Hali:
10 12 3 4 6 12 4 7 8 9
En Büyük Eleman İndisi: 1 ve Eleman 12
En Küçük Eleman İndisi: 2 ve Eleman 3
Dizinin SON Hali:
10 15 3 4 6 12 4 7 8 9
*/

```

Aynı program dizi nesnesinin yöntemleriyle aşağıdaki gibi yazılabilir;

```

int[] dizi = { 10, 12, 3, 4, 6, 12, 4, 7, 8, 9 };
Console.WriteLine("Dizinin İLK Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");
Console.WriteLine();
int enKucuk = dizi.Min();
int enBuyuk = dizi.Max();
/* İlk elemanlar hem en küçük hem de en büyük olsun. */
int enKucukIndex = dizi.Single(enKucuk);
int enBuyukIndex = dizi.Single(enBuyuk);
Console.WriteLine($"En Büyük Eleman İndisi: {enBuyukIndex} ve Eleman {enBuyuk}");

```

```

Console.WriteLine($"En Küçük Eleman İndisi: {enKucukIndex} ve Eleman {enKucuk}");
//En büyük elemana en küçüğünü ekleme
dizi[enBuyukIndex] += dizi[enKucukIndex];
Console.WriteLine("Dizinin SON Hali:");
for (int indis = 0; indis < dizi.Length; indis++)
    Console.Write($"{dizi[indis],3}");

```

7.5 Dizileri Kopyalama

Dizinin bir kısmı bir başka diziye `Copy()` yöntemiyle kopyalanabilir;

```

var kaynak1 = new int[] { 11, 12, 3, 5, 2, 9, 28, 17 };
var hedef1 = new int[3];
Array.Copy(kaynak1, hedef1, 3); // 11, 12, 3

```

Kaynak dizinin bir kısmı hedef dizinin bir bölümüne `CopyTo()` yöntemiyle kopyalanabilir;

```

var kaynak2 = new int[] { 11, 12, 7 };
var hedef2 = new int[6];
kaynak2.CopyTo(hedef2, 2); // 0, 0, 11, 12, 7, 0

```

Bir dizinin özdeşi `Clone()` yöntemiyle yapılabilir;

```

var kaynak3 = new int[] { 11, 12, 7 };
var hedef3 = kaynak3.Clone(); // 11,12,17

```

7.6 Dizileri Karşılaştırma

Diziler, dizi nesnesinin `SequenceEqual()` yöntemi ile karşılıklı elemanlar karşılaştırılabilir.

```

int[] dizi1 = { 3, 5, 7 };
int[] dizi2 = { 3, 5, 7 };
bool sonuc = dizi1.SequenceEqual(dizi2);
Console.WriteLine("İki Dizi Eşit Mi? {0}", sonuc); //true

```

7.7 Array Sınıfı Yöntemleri

Yukarıda birkaç yöntemi açıklanan `Array` sınıfının yöntemleri aşağıda verilmiştir;

Yöntem	Açıklama	Örnek Kullanım
<code>Sort()</code>	Dizi elemanlarını küçükten büyüğe (veya alfabetik) sıralar.	<code>Array.Sort(sayilar);</code>
<code>Reverse()</code>	Dizinin mevcut eleman sıralamasını tam tersine çevirir.	<code>Array.Reverse(sayilar);</code>
<code>Clear()</code>	Belirlenen aralıktaki elemanları varsayılan değerine (0, null vb.) çeker.	<code>Array.Clear(sayilar, 0, 2);</code>
<code>IndexOf()</code>	Belirli bir değer dizideki ilk konumunu (indisini) döndürür.	<code>int sira = Array.IndexOf(dizi, "C#");</code>
<code>LastIndexOf()</code>	Belirli bir değer dizideki son konumunu döndürür.	<code>Array.LastIndexOf(dizi, 5);</code>
<code>Copy()</code>	Bir dizinin bir kısmını veya tamamını başka bir diziye kopyalar.	<code>Array.Copy(kaynak, hedef, 3);</code>
<code>Resize()</code>	Dizinin boyutunu yeniden ayarlar (Arka planda yeni bir dizi oluşturur).	<code>Array.Resize(ref dizi, 10);</code>
<code>BinarySearch()</code>	Sıralanmış bir dizide ikili arama yaparak değer indisini bulur.	<code>int konum = Array.BinarySearch(dizi, 42);</code>
<code>Find()</code>	Belirli bir koşula uyan ilk elemanı döndürür.	<code>var sonuc = Array.Find(dizi, x => x > 5);</code>
<code>Exists()</code>	Dizide belirli bir koşulu sağlayan eleman olup olmadığını kontrol eder.	<code>bool varMi = Array.Exists(dizi, x => x < 0);</code>

Tablo 7.1: Array Sınıfı Yöntemleri

7.8 Çok Boyutlu Diziler

Su ana kadar gördüğümüz tek boyutlu dizilerdi. **İki boyutlu diziler** (**two-dimensional array**) matematikten de bildiğimiz üzere matris (**matrix**) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir. Matris işlemleri gibi bazı problemlerde; bir dizinin her bir elemanının da dizi olması istenir. Bu tür iki boyutlu dizilerde en içteki dizinin boyutu kimliklendirmede sağda yer alır.

```
// 2 satır ve 3 sürundan oluşan iki boyutlu tam sayı dizisi aşağıdaki gibi tanımlanır;
int[,] çokboyutludizi = new int[2, 3];
// Eleman değerleri bilinen bir iki boyutlu dizinin başlatılması aşağıdaki gibi yapılır;
int[,] çokboyutludizi1 = new int[2, 3] { { 1, 2, 3 }, { 4, 5, 6 } };
int[,] çokboyutludizi2 = { { 1, 2, 3 }, { 4, 5, 6 } };
```

Örnek olarak 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
int[,] matris = new int[3, 4]; // 3 satır 4 sütun
int satır, sütun; // indis değişkenleri
/*
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[0,sütun] = Convert.ToInt32(Console.ReadLine());
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[1,sütun] = Convert.ToInt32(Console.ReadLine());
for (sütun=0; sütun<matris.GetLength(1); sütun++) //Birinci Satır Okuyalım
    matris[2,sütun] = Convert.ToInt32(Console.ReadLine());
//Bunun yerine iç içe iki for tanımlanır;
*/
for (satır = 0; satır < matris.GetLength(0); satır++)
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        matris[satır, sütun] = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Okunan Matris:");
for (satır = 0; satır < matris.GetLength(0); satır++) {
    for (sütun = 0; sütun < matris.GetLength(1); sütun++) //Birinci Boyut İçin
        Console.Write($"{matris[satır, sütun],4}");
    Console.WriteLine();
}
/*
11
12
13
14
21
22
23
24
31
32
33
34
Okunan Matris:
11 12 13 14
21 22 23 24
31 32 33 34
*/
```

Bir başka örnek olarak elemanları belirli 5x4'lük matrisin satır ve sütun toplamalarını ekrana yazan programı verilebilir;

```
int[,] matris = { {10,20,30},
                  {40,50,60},
                  {70,80,90},
                  {15,25,35},
                  { 5,15,25} };
int satır, sütun;
Console.WriteLine("Matris:");
```

```

for (satır = 0; satır < matris.GetLength(0); satır++) {
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        Console.WriteLine($"{matris[satır, sütun],4}");
    Console.WriteLine();
}
Console.WriteLine("Satır Toplamları:");
for (satır = 0; satır < matris.GetLength(0); satır++) {
    int satırToplam = 0;
    for (sütun = 0; sütun < matris.GetLength(1); sütun++)
        satırToplam += matris[satır, sütun];
    Console.WriteLine($" {satırToplam,4}");
}
Console.WriteLine();
Console.WriteLine("Sütun Toplamları:");
for (sütun = 0; sütun < matris.GetLength(1); sütun++) {
    int sütunToplam = 0;
    for (satır = 0; satır < matris.GetLength(0); satır++)
        sütunToplam += matris[satır, sütun];
    Console.WriteLine($" {sütunToplam,4}");
}
/*
Matris:
10 20 30
40 50 60
70 80 90
15 25 35
5 15 25
Satır Toplamları: 60 150 240 75 45
Sütun Toplamları: 140 190 240
*/

```

Çok boyutlu dizi (multi-dimentional array) örneği olarak şöyle bir örnek verilebilir; Üç farklı dersten (1.boyut) dört değişik not (2.boyut) alan 10 öğrencili (3.boyut) bir sınıf vardır. Notların ağırlıkları %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan programı aşağıda verilmiştir;

```

int[, ,] notlar = new int[3, 4, 10]; //DERSSAYISI=3, SINAVSAYISI=4, ÖĞRENCİSAYISI=10
int i, j, k; //indis değişkenleri sırasıyla i:ders, j:sınav, k:öğrenci
int[] agirlik = { 10, 30, 20, 40 }; //Odevler:%10, Vize:%30, Hızlı Sınav:%20, Final:%40
Random rastgele = new Random();
//Console.WriteLine("Notlar:");
for (i = 0; i < notlar.GetLength(0); i++) { //derslerdeki (3. boyut)
    //Console.WriteLine($"{i}. Ders: ");
    for (j = 0; j < notlar.GetLength(1); j++) { //sınavlara giren (2. boyut)
        //Console.WriteLine($"{j}. Sınav:");
        for (k = 0; k < notlar.GetLength(2); k++) { //öğrencilerin (1. boyut)
            int rastgeleNot = rastgele.Next(0, 100); //Notlarını Rastgele Not Veriliyor!
            notlar[i, j, k] = rastgeleNot;
            //Console.WriteLine($"{rastgeleNot} ");
        }
        //Console.WriteLine();
    }
}
Console.WriteLine("Sınav Ortalamaları:");
for (i = 0; i < notlar.GetLength(0); i++) { //derslerdeki (3. boyut)
    Console.WriteLine($"{i}.Ders:");
    for (j = 0; j < notlar.GetLength(1); j++) { //sınavlara giren (2. boyut)
        Console.WriteLine($"{j}.Sınav: ");
        int notToplam=0;
        for (k = 0; k < notlar.GetLength(2); k++) //öğrencilerin Notları (1. boyut)
            notToplam += notlar[i, j, k];
        float ortalama=notToplam / notlar.GetLength(2);
        Console.WriteLine($"{ortalama:F2} ");
    }
}

```



```

    }
    Console.WriteLine();
}
Console.WriteLine("Her bir Öğrencinin Ağırlıklı Ortalaması:");
for (k = 0; k < notlar.GetLength(2); k++) { //öğrencilerin (1. boyut)
    Console.WriteLine($"{k}.Öğrenci: ");
    for (i = 0; i < notlar.GetLength(0); i++) { //derslerdeki (3. boyut)
        Console.WriteLine($"{i}.Ders: ");
        float agirlikliNot = 0.0f;
        for (j = 0; j < notlar.GetLength(1); j++) { //sınavları (2. boyut)
            agirlikliNot += notlar[i, j, k] * agirlik[j] / 100.0f;
            Console.WriteLine($"{agirlikliNot:F1} ");
        }
    }
    Console.WriteLine();
}
}
/*Program Çıktısı:
Sınav Ortalamaları:
0.Ders: 0.Sınav: 34.00 1.Sınav: 54.00 2.Sınav: 56.00 3.Sınav: 45.00
1.Ders: 0.Sınav: 50.00 1.Sınav: 49.00 2.Sınav: 46.00 3.Sınav: 46.00
2.Ders: 0.Sınav: 56.00 1.Sınav: 36.00 2.Sınav: 43.00 3.Sınav: 66.00
Her bir Öğrencinin Ağırlıklı Ortalaması:
0.Öğrenci: 0.Ders: 2.5 18.1 29.1 39.9 1.Ders: 5.6 17.3 30.7 47.5 2.Ders: 2.0 18.8 30.4 41.2
1.Öğrenci: 0.Ders: 5.2 32.5 46.3 79.9 1.Ders: 8.5 11.5 28.1 32.5 2.Ders: 7.8 14.7 22.3 59.9
2.Öğrenci: 0.Ders: 1.4 21.2 32.2 51.0 1.Ders: 2.2 12.7 17.1 38.7 2.Ders: 8.4 24.6 31.4 51.0
3.Öğrenci: 0.Ders: 8.6 12.2 27.6 46.0 1.Ders: 6.0 19.5 20.7 41.1 2.Ders: 1.1 21.5 40.5 78.1
4.Öğrenci: 0.Ders: 2.8 28.9 37.1 37.9 1.Ders: 5.9 34.4 44.0 77.6 2.Ders: 2.9 5.3 12.7 29.9
5.Öğrenci: 0.Ders: 0.5 12.8 28.2 53.0 1.Ders: 2.0 11.9 16.7 25.5 2.Ders: 7.8 12.9 13.9 36.7
6.Öğrenci: 0.Ders: 1.2 24.6 34.6 63.0 1.Ders: 7.4 32.9 38.5 38.9 2.Ders: 9.3 14.1 25.3 60.1
7.Öğrenci: 0.Ders: 1.4 17.3 30.5 46.5 1.Ders: 2.8 11.2 26.8 48.0 2.Ders: 6.0 17.7 24.1 59.7
8.Öğrenci: 0.Ders: 7.8 12.9 22.7 31.9 1.Ders: 3.2 12.5 17.5 55.9 2.Ders: 1.6 6.7 11.3 41.3
9.Öğrenci: 0.Ders: 3.5 16.7 22.7 42.7 1.Ders: 7.2 35.4 51.8 73.4 2.Ders: 9.1 29.8 40.6 59.0
*/

```

7.9 Pürüzlü Diziler

Pürüzlü diziler (jagged array) dizilerin dizisi olarak adlandırılabilir. Buradaki her dizi farklı boyutta olabilir.

```

// Pürüzlü (jagged) dizi
int[][] pürüzlüDizi = new int[5][]; // 5 adet dizinin dizisi
pürüzlüDizi[0] = new int[3] { 1, 2, 3 }; //birinci dizi 3
pürüzlüDizi[1] = new int[] { }; // ikinci dizi 0 elemanlı
pürüzlüDizi[2] = new int[2] { 4, 5 }; // üçüncü dizi 2 elemanlı
pürüzlüDizi[3] = new int[1] { 6 }; // dördüncü dizi 1 elemanlı
pürüzlüDizi[4] = new int[5] { 7, 8, 9, 10, 11 }; // beşinci dizi 5 elemanlı
Console.WriteLine("Pürüzlü Dizi:");
for (int i = 0; i < pürüzlüDizi.Length; i++) {
    for (int j=0; j < pürüzlüDizi[i].Length; j++)
        Console.WriteLine($"{0,4}",pürüzlüDizi[i][j]);
    Console.WriteLine();
}

```

7.10 Foreach Talimatı

Yalnızca bir koleksiyondaki öğeler arasında döngü oluşturmak için kullanılan **foreach döngüsü (foreach loop)**, **aralık tabanlı döngü (range based loop)** olarak da bilinir. Bu döngü ileride göreceğimiz bütün koleksiyonlarda kullanılır. Aşağıdaki gibi yazılır;

```
foreach (veritipi dizi-elemanını-temsil-eden-değişken in dizi-kimliği)
    döngü-kodu;
```

foreach, bir koleksiyonun (dizi, liste vb.) her bir öğesi için bir kez çalışan, yönetimi tamamen C# derleyicisine bırakılmış bir döngü yapısıdır. Buna örnek olarak aşağıda bir dizi programı örneği verilebilir;


```
int[] dizi = { 1, 2, 3, 4, 5 };
float toplam = 0.0f;
foreach (int eleman in dizi)
    toplam += eleman;
float ortalama = toplam / dizi.Length;
Console.WriteLine("Dizi Ortalaması:{0}", ortalama);
```

§7.10.1 Foreach Neden Tercih Edilir?

- ♦ **Güvenlik:** `for` döngüsünde dizinin sınırlarını aşma (`IndexOutOfRangeException`) riski varken, `foreach` ile bu risk yoktur. C# dizinin sonuna geldiğini kendisi bilir.
- ♦ **Okunabilirlik:** İndis değişkenleri (`i`, `j`) ile uğraşmazsınız. Kod, "Koleksiyondaki her bir öğe için şunu yap" şeklinde bir cümle gibi okunur.
- ♦ **Soyutlama:** Sadece dizilerle değil, ileride göreceğimiz `List`, `Dictionary`, `Stack` gibi tüm `IEnumerable` arayüzünü uygulayan yapılarla aynı şekilde çalışır.

Foreach Döngüsü

`foreach` döngüsü içerisinde, **o an üzerinde gezindiğiniz değişkenin değerini değiştiremezsiniz!**

```
int[] sayilar = { 1, 2, 3 };
foreach (int s in sayilar)
{
    // s = s * 2; // HATA! Döngü değişkenine atama yapılamaz.
}
```

§7.10.2 IEnumerable ve IEnumerator

Derleyici `foreach` gördüğünde onu bir `while` döngüsüne çevirir. Aslında arka planda şu işlemler döner:

- ♦ Koleksiyondan bir `GetEnumerator()` istenir.
- ♦ `MoveNext()` metodu ile bir sonraki öğe var mı bakılır.
- ♦ `Current` özelliği ile o anki değer okunur.

Bunun detayı için ileride anlatacağımız Yineleyici desenini inceleyebilirsiniz.

`foreach` döngüsü ile bir listenin içinde gezerken, listeden bir öğeyi silemeyiz (`liste.Remove(öge)`). C# çalışma zamanı (**run-time**), bir koleksiyon `foreach` ile taranırken eleman ekleme ve çıkarma gibi koleksiyonun yapısının değiştirilmesine izin verilmez ve `InvalidOperationException` hatası fırlatılır.

7.11 Dizilerde Dilimleme

Geleneksel programlamada bir dizinin son elemanına erişmek veya belirli bir parçasını kopyalamak için `array.Length-1` gibi matematiksel hesaplamalar gerekirdi. C#8.0+ ile gelen `Index` ve `Range` yapıları, bu işlemleri doğal bir söz dizimi ile gerçekleştirmenize olanak tanır.

§7.11.1 Index Yapısı

`System.Index` yapısı, bir dizinin başından veya sonundan bir konumu temsil eder. Buradaki en kritik araç, sapka işleci (hat operator) olarak da bilinen (^) işaretidir;

- ♦ ^1: Dizinin son elemanını (`Length-1`) temsil eder.
- ♦ ^0: Dizinin uzunluğunu temsil eder (doğrudan indisleme için kullanıldığında hata verir, ancak aralıklarda bitiş belirtmek için kullanılır).

```
string[] diller = { "C", "C++", "C#", "Java", "Python" };
// Eski yöntem
string sonDilEski = diller[diller.Length - 1];
// Modern yöntem
string sonDilYeni = diller[^1]; // Python
string sondanIkincisi = diller[^2]; // Java
```

§7.11.2 Range Yapısı

`System.Range` yapısı, bir dizinin başlangıç ve bitiş indislerini belirterek bir **dilim** (**slice**) almanızı sağlar. **Aralık işleci** (**range operator**) olarak bilinen `..` işleci ile işlem yapılır.

```
int[] sayilar = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
// 1. indisten başla, 4. indise kadar git (4 dahil değil)
int[] altDizi = sayilar[1..4]; // { 1, 2, 3 }
```

```
// İlk 3 elemanı al
int[] ilkUc = sayilar[..3]; // { 0, 1, 2 }
// 7. indisten sona kadar al
int[] sonUc = sayilar[7..]; // { 7, 8, 9 }
// Tüm diziyi kopyala
int[] kopya = sayilar[..];
```

7.12 Düşün ve Çöz

- ◊ **Düşün:** C#'ta dizi ile `List<T>` arasındaki temel farklar nelerdir? Sabit boyutlu bir dizi yerine liste kullanmak her zaman daha iyi midir? Hangi senaryolarda dizi tercih edilmelidir?
- ◊ **Düşün:** Çok boyutlu dizi (**multidimensional array**) ile pürüzlü dizi (**jagged array**) arasındaki farkı bellek kullanımı açısından açıklayınız. Bir matris hesaplaması için hangisi daha uygundur?
- ◊ **Düşün:** `foreach` döngüsü, `for` döngüsüne kıyasla bir dizi üzerinde yinelemede hangi avantajları sağlar? `foreach` içinde dizinin eleman değerini değiştirmeye çalışırsanız ne olur ve neden?
- ◊ **Düşün:** Bir dizinin sıralanması (**sorting**) için kabarcık sıralama (**bubble sort**) ile `Array.Sort()` karşılaştırıldığında performans ve kullanım kolaylığı açısından hangisi tercih edilmelidir? Kendi sıralama algoritmanızı yazmayı ne zaman düşünürsünüz?
- ◊ **Düşün:** Dizi sınırı dışı erişim (`IndexOutOfRangeException`) C#'ta en sık karşılaşılan hatalardan biridir. Bu hatayı önlemenin hem tasarım hem de kod yazım aşamasındaki yolları nelerdir?
- ◊ **Düşün:** `IEnumerable<T>` arayüzü diziler ve koleksiyonlar için neden temel bir soyutlama olarak kabul edilir? `foreach` döngüsünün arka planda bu arayüzü nasıl kullandığını açıklayınız.
- ◊ **Düşün:** Dizileri kopyalarken yüzeysel kopya (**shallow copy**) ile derin kopya (**deep copy**) arasındaki farkı referans tipler içeren bir dizi üzerinden açıklayınız. `Array.Copy()` hangi türde kopya yapar?
- ◊ **Çöz:** 10 elemanlı bir `int` dizisi tanımlayın, elemanları rastgele 1-100 arasında doldurun ve kabarcık sıralama (**bubble sort**) algoritmasıyla küçükten büyüğe sıralayın. Her adımı konsola yazdırın.
- ◊ **Çöz:** Kullanıcıdan N adet not alan (N kullanıcı tarafından girilecek) ve bu notların ortalamasını, standart sapmasını, en yüksek ve en düşük notu hesaplayıp ekrana yazan programı yazınız.
- ◊ **Çöz:** Bir 3x3 matris (çok boyutlu dizi) tanımlayıp elemanlarını kullanıcıdan alın. Matrisin transpozunu hesaplayıp ekrana yazdırın.
- ◊ **Çöz:** Bir `string` dizisi içinde aynı ismin kaç kez geçtiğini bulan; 'İlhan', 'Ahmet', 'Mehmet', 'İlhan', 'Ahmet', 'İlhan' gibi bir dizide her ismin tekrar sayısını gösteren programı yazınız.
- ◊ **Çöz:** İki adet sıralı `int` dizisini tek bir sıralı dizide birleştiren (**merge**) algoritmasını `Array.Sort()` kullanmadan yazınız. Zaman karmaşıklığını belirtiniz.
- ◊ **Düşün:** Bir dizinin boyutunu program çalışırken neden `sayilar.Length = 10;` diyerek değiştiremiyoruz? Eğer daha büyük bir diziye ihtiyacımız olursa ne yapmalıyız?
- ◊ **Düşün:** `foreach` döngüsüyle bir dizi üzerinde dönerken dizi elemanlarını değiştiremiyoruz; ancak `for` döngüsüyle değiştirebiliyoruz. Bu kısıtlamanın temel sebebi (`IEnumerable` mantığıyla) nedir?
- ◊ **Düşün:** Elinizde bir sınıfın öğrenci isimlerini tutan bir dizi var. Bu diziyi alfabetik sıraya dizmek için C#'ın hangi hazır metodunu kullanırsınız?